

데이터베이스

Database

데이터를 정의하는 SQL

- ◆ 이전 시간에 데이터를 제어하는 명령어들(**DML**)에 대해서 알아보았습니다.
- ◆ 이어서 제약조건에 대해서 이야기하기로 했는데 이를 위해서 데이터를 정의하는 명령어(**DDL**)에 대해서 알아보면서, 이에 활용할 수 있는 제약조건에 대해 알아보겠습니다.
- ◆ 데이터 정의어에는, **Data(Table)**를 만들고 제거하고 수정하는 명령어들입니다.
- ◆ 그런데 일전에 사용한 명령어가 아닌 독립적 명령어를 사용합니다.
- ◆ 중요한 **Data**를 다루는 만큼 실수와 오류를 막고, 더불어 접근을 제한하기 위해서 입니다.

데이터를 정의하는 SQL : CREATE

◆ 이미 앞서 예시를 통해서 만난 CREATE문

◆ DB 생성

- CREATE DATABASE {DB명};
- CREATE DATABASE TEST;

◆ Table 생성

- CREATE TABLE {Table명} (속성명 속성타입 [제약조건], ..., [CONSTRAINT {제약조건}])
- CREATE TABLE DEPT (DEPTNO DECIMAL(2), DNAME VARCHAR(14), LOC VARCHAR(13), CONSTRAINT PK_DEPT PRIMARY KEY (DEPTNO));

데이터를 정의하는 SQL : CREATE

◆ 제약조건(Constraint, 제약)

: 특정 속성에 추가적으로 부여하는 조건으로 이를 충족하지 못하면 생성되지 못하도록 걸어두는 일종의 제약으로, 오류를 줄이고 잘못된 **DATA**가 입력되는 경우를 없애준다. 또한, 특정 제약을 통해서 성능에 도움을 줄 수 있다.

◆ 제약조건 작성 방법

: 속성을 작성할 때 바로 뒤에 붙여 제약조건을 붙이거나 모든 속성을 작성한 다음 '**CONSTRAINT**'문을 이용하여 제약조건 이름과 함께 제약조건을 부여

데이터를 정의하는 SQL : 제약조건

◆ NOT NULL

: NULL 값을 허용하지 않는 제약조건. 속성 뒤에 바로 이어서 작성한다.

```
CREATE TABLE TEMP (ID INT NOT NULL, NAME VARCHAR(15));
```

◆ DEFAULT

: 미입력 혹은 'DEFAULT'를 작성하면 기본값으로 들어가는 값을 지정한다. 속성 뒤에 바로 이어서 작성한다.

```
CREATE TABLE TEMP (ID INT NOT NULL, NAME VARCHAR(15)  
DEFAULT 'unknown');
```

◆ 이 둘의 경우에는 CONSTRAINT문을 이용한 추가를 할 수 없다.

데이터를 정의하는 SQL : 제약조건

◆ UNIQUE

: 중복을 허용치 않는다. 이미 존재하는 값을 입력시 적용되지 않도록 합니다.

```
CREATE TABLE TEMP (ID INT NOT NULL UNIQUE, NAME  
VARCHAR(15) DEFAULT 'unknown');
```

/or/

```
CREATE TABLE TEMP (ID INT NOT NULL, NAME VARCHAR(15)  
DEFAULT 'unknown', CONSTRAINT UNI_TEMP UNIQUE (ID));
```

데이터를 정의하는 SQL : 제약조건

◆ CHECK

: 사전에 조건을 주어 참인 경우에만 허용한다.

```
CREATE TABLE TEMP (ID INT NOT NULL UNIQUE CHECK (ID>0),  
NAME VARCHAR(15) DEFAULT 'unknown');
```

/or/

```
CREATE TABLE TEMP (ID INT NOT NULL UNIQUE, NAME  
VARCHAR(15) DEFAULT 'unknown', CONSTRAINT CH_TEMP CHECK  
(ID>0));
```

데이터를 정의하는 SQL : 제약조건

◆ PRIMARY KEY(기본키)

: Table의 행(Tuple)을 유일하게 가리킬 수 있게 해주는 'Key'를 지정하는 제약조건. 이 Tuple이 식별되도록 기준이 되는 속성이여야 하며, 유일성과 최소성을 만족해야 한다.

```
CREATE TABLE TEMP (ID INT PRIMARY KEY CHECK (ID>0),  
NAME VARCHAR(15) DEFAULT 'unknown');
```

/or/

```
CREATE TABLE TEMP (ID INT CHECK (ID>0), NAME  
VARCHAR(15) DEFAULT 'unknown', CONSTRAINT PK_TEMP  
PRIMARY KEY (ID));
```

◆ NOT NULL과 UNIQUE는?

: 이미 PRIMARY KEY상에 이 두 기능이 기본으로 포함되어 있다.

데이터를 정의하는 SQL : 제약조건

◆ FOREIGN KEY(외래 키)

: 다른 Table의 기본키. 해당 Table의 유일성을 유지해주지는 않으나 Table간의 관계를 형성할 때 조건으로 사용하여 무결성을 유지시켜 준다.

```
CREATE TABLE TEMP_FK (ID INT PRIMARY KEY, PID INT  
FOREIGN KEY(PID) REFERENCES TEMP(ID));
```

/or/

```
CREATE TABLE TEMP_FK (ID INT PRIMARY KEY , PID INT,  
CONSTRAINT FK_TEMP FOREIGN KEY(PID) REFERENCES  
TEMP(ID));
```

◆ 관계 테이블의 기본키가 가지지 않은 값은 외래 키 값으로 넣을 수 없음.

데이터를 정의하는 SQL : 제약조건

- ◆ 외래키 추가 조건
: 앞선 것과 같이 관계 테이블의 기본키가 가지고 있지 않은 값이라면 넣을 수 없는 조건을 가지고 있기 때문에 관계 테이블 상에 삭제, 수정 간에 문제가 발생할 수 있음.
- ◆ 사라진 기존의 값
: 삭제, 수정 과정에서 외래키의 관계에 해당하는 값이 사라져 더 이상 외래키로서 역할을 수행할 수 없는 경우 4가지 제약을 걸어둘 수 있다.

데이터를 정의하는 SQL : 제약조건

- ◆ ON DELETE [RESTRICT | CASCADE | NO ACTION | SET NULL]
ON UPDATE [RESTRICT | CASCADE | NO ACTION | SET NULL]
RESTRICT : 외래키의 관계에 해당하는 개체가 사라질 수 있는 경우 이를 사전에 경고, 막는다.
CASCADE : 외래키의 관계에 해당하는 개체가 사라질 경우 이와 관련된 외래키를 가진 데이터를 같이 변경, 삭제된다.
NO ACTION : 문제가 발생하더라도 무시한다.
SET NULL : 외래키의 관계에 해당하는 개체가 사라질 경우 이와 관련된 외래키를 가진 데이터를 모두 NULL로 대체한다. 이 경우 NOT NULL과 같이 사용할 수 없다.
CREATE TABLE TEMP_FK (ID INT PRIMARY KEY, PID INT,
CONSTRAINT FK_TEMP FOREIGN KEY(PID) REFERENCES TEMP(ID)
ON DELETE CASCADE);

데이터를 정의하는 SQL : DELETE

◆ DB 제거

- DROP DATABASE {DB명};
- DROP DATABASE TEST;
- DB를 지우고 만드는 것은 매우 주의해야한다.

◆ TABLE 제거

- DROP TABLE {Table명};
- DROP TABLE TEST;

데이터를 정의하는 SQL : CREATE/UPDATE

◆ Table에 칼럼 추가

- ALTER TABLE {Table명} ADD COLUMN {Column명} {Datatype} [DEFAULT {기본값}] [제약조건] ;
- ALTER TABLE TEMP ADD COLUMN AGE INT DEFAULT 20 CHECK (AGE>0);
: 추가된 속성에 대해서 DEFAULT를 설정하지 않는다면 모두 NULL 값으로 들어가게 된다.
- ALTER TABLE TEMP ADD COLUMN PHONE_NUMBER VARCHAR(17) UNIQUE;

◆ Table에 제약조건 추가

- ALTER TABLE {Table명} ADD CONSTRAINT {제약조건명} {제약조건};
- ALTER TABLE TEMP ADD CONSTRAINT UNI_PN UNIQUE(PHONE_NUMBER);

데이터를 정의하는 SQL : CREATE/UPDATE

◆ Table의 속성 제거

- ALTER TABLE {Table명} DROP COLUMN {Column명};
- ALTER TABLE TEMP DROP COLUMN AGE;
- ALTER TABLE TEMP DROP CONSTRAINT UNI_PN;

◆ Table의 속성 변경

- ALTER TABLE {Table명} MODIFY COLUMN {Column명} {바뀔 타입} {바뀔 제약조건};
- ALTER TABLE TEMP MODIFY COLUMN AGE INT NOT NULL;
: NOT NULL의 경우 기본 설정이 NULL이 가능한 상태이므로 ADD가 아닌 MODIFY를 사용한다.
- ALTER TABLE TEMP MODIFY COLUMN AGE INT UNSIGNED;

Primary Key와 Foreign Key

- ◆ 아마 기본키(Primary Key)와 외래키(Foreign Key)에 대해서 이해가 잘 되지 않았을 것입니다.
- ◆ 왜냐면 이것은 '완성된 개념'이기 때문입니다.
- ◆ 굳이 우리가 '태양'을 '우리 행성계 질량의 99.8%를 차지하며, 중력으로 주변 천체들을 붙들고 있는 거대한 플라스마 구체이며, 중심부의 압도적인 중력이 수소 원자들을 짓눌러 하나로 합쳐지게 만들며(핵융합), 이 과정에서 소실되는 질량이 엄청난 양의 빛과 열에너지로 변환되어 우주로 뿔어나가는 우리가 '항성'이라고 부르는 것'이라고 표현하지 않는 것과 같습니다.
- ◆ 즉, 이는 '인간의 제한적 뇌 용량을 극도로 활용하기 위한 추상화' 과정인 셈입니다.
- ◆ 그러니 우리가 이 '이름의 의미'에 대해서 깊게 생각할 필요는 없습니다.
- ◆ 어차피 구현은 다 되어 있으니깐요.

Primary Key와 Foreign Key

- ◆ 그럼 이 둘을 어떤 것이고 어떻게 사용하는게 바른 방법일까요?
- ◆ 우선 기본키는 'ID'입니다.
- ◆ ID는 유일해야하며, 그 이유는 우리가 이 데이터를 '식별'하기 위함입니다.
- ◆ 그리고 그 속성을 유지하기 위해 복잡한 과정을 거쳐야합니다.
- ◆ 다만, 기본키로 지정하면 이 모든 과정이 '자동'이 됩니다.
- ◆ 기본키는 그렇게 이용합니다.
- ◆ 그럼 외래키는 무엇일까요?
- ◆ 외래키는 기본적으로 '다른 데이터(테이블)에서 기본키인 것'입니다.
- ◆ 아니, 기본키만 있으면 되지 왜 외래키가 있느냐?
- ◆ 그 중심에 '관계'가 존재합니다.

Primary Key와 Foreign Key

- ◆ 우리가 사용하는 MariaDB와 더불어서 현존 대부분의 일반적인 DB는 'RDBMS(Relational DBMS)'로, '관계형 데이터베이스'입니다.
- ◆ 즉, 데이터(테이블)과 데이터(테이블) 간의 '관계'를 중심으로 하는 데이터베이스라는 것입니다.
- ◆ 이는 과거 DB들이 '연산 속도에 비해, 저장공간이 부족하다!'는 상황에서 기반된 것이고, 이후에 개선과 발전을 거친 후에도 여전히 사용성이 좋기에 현재 대부분의 DB의 형태로 자리 잡은 것입니다.
- ◆ 즉, 외래키란, 그 관계의 연장선에 있는 중요한 규칙입니다.
- ◆ 그리고 그 규칙을 '지켜야될 제약'으로서 본 표현이 '제약조건'인 셈입니다.
- ◆ 즉, 내가 '이 데이터와 이 데이터가 연관이 있다!'를 표현하면서, 관계를 통해 자원을 아끼고자 한다면 외래키를 활용하는 것입니다.

Primary Key와 Foreign Key

- ◆ 다만 과유불급이라고, 관계가 지나치게 많아지면 오히려 '자원은 충분한 시대에 너무 많은 연산이 아니야?'라는 역전 현상이 생길 수도 있습니다.
- ◆ 실제 본인도 '왜 외래키들을 이용하여 잘게 쪼개놓지 않았지?'라고 생각한 현장에서의 **DB**를 보고 개발자에게 질문하니 돌아온 질문은 '그게 더 복잡해질 수 있다'라는 답변이었습니다.
- ◆ 현실과 실무의 괴리도 이해해야합니다.
- ◆ 다만 그러려면 경험이 무조건적으로 필요합니다.
- ◆ 때문에 우선 우리는 '잘게 쪼개기부터' 시도 하거나 '일체화 하여 관리하기' 둘 중 하나부터 시도해보며 그러지 않았을 때의 이점을 고민해보는 시간을 가지며 나아가보도록 하겠습니다.
- ◆ 이런 구조와 설계에 대한 부분은 차후에 더 다루도록 하겠습니다.